



**Министерство науки и высшего образования Российской
Федерации**
**Федеральное государственное бюджетное образовательное
учреждение высшего образования**
**«Московский государственный технический университет
имени Н.Э. Баумана**
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 6
по дисциплине «Анализ алгоритмов»

Тема Методы решения задачи коммивояжера

Студент Пантелеев Василий Максимович

Группа ИУ7-52Б

Преподаватели Строганов Д.В. Волкова Л.Л.

Москва, 2024

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
ВВЕДЕНИЕ	4
1 Аналитическая часть	5
1.1 Алгоритм полного перебора	5
1.2 Муравьиный алгоритм	5
2 Конструкторская часть	7
2.1 Описание алгоритмов	7
2.2 Модель вычислений	14
2.3 Вычисление трудоемкости алгоритмов	14
2.3.1 Алгоритм полного перебора	15
2.3.2 Муравьиный алгоритм	15
3 Технологическая часть	17
3.1 Средства реализации	17
3.2 Реализация алгоритмов	18
3.3 Тестирование	21
4 Исследовательская часть	23
4.1 Технические характеристики ЭВМ	23
4.2 Сравнение алгоритмов	23
4.3 Параметризация	24
4.3.1 Класс данных	24
4.3.2 Результат параметризации	27
ЗАКЛЮЧЕНИЕ	29
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	30
ПРИЛОЖЕНИЕ А	31

ПРИЛОЖЕНИЕ А	31
-------------------------------	-----------

ВВЕДЕНИЕ

Задача коммивояжера (Traveling Salesman Problem, TSP) — это классическая задача комбинаторной оптимизации. Её цель — найти маршрут, проходящий через заданное множество городов ровно один раз и минимизирующий общее расстояние или стоимость пути[1].

Пусть дано множество городов $V = \{v_1, v_2, \dots, v_n\}$ и матрица расстояний $D = [d_{ij}]$, где d_{ij} — расстояние между городами v_i и v_j . Требуется найти перестановку $P = (p_1, p_2, \dots, p_n)$, минимизирующую суммарное расстояние:

$$\min \sum_{k=1}^{n-1} d_{p_k p_{k+1}},$$

где каждый город посещается ровно один раз. Для замкнутого маршрута дополнительно учитывается возврат в начальный город: $d_{p_n p_1}$.

Цель лабораторной работы — выполнить сравнительный анализ метода полного перебора решения задачи коммивояжера с методом на базе муравьиного алгоритма. Для достижения поставленной цели необходимо выполнить следующие задачи:

- описать стандартный метод решения задачи коммивояжера и метод на базе муравьиного алгоритма;
- описать модель вычислений;
- выполнить оценку трудоемкости разрабатываемых алгоритмов;
- реализовать разработанные алгоритмы;
- провести сравнительный анализ двух рассмотренных методов решения задачи коммивояжера;
- выполнить параметризацию метода, основанного на муравьином алгоритме по трём его параметрам.

1 Аналитическая часть

В данном разделе будут рассмотрены теоретические аспекты алгоритмов решения задачи коммивояжера.

1.1 Алгоритм полного перебора

Алгоритм полного перебора для задачи коммивояжера основывается на генерации всех возможных маршрутов, которые проходят через заданное множество городов ровно один раз. Каждый маршрут представляется как перестановка множества городов.

Процесс решения включает два этапа:

- 1) генерация всех перестановок множества городов;
- 2) для каждой перестановки вычисление общей длины маршрута на основе заданной матрицы расстояний и выбор маршрута с минимальной длиной.

Алгоритм гарантирует нахождение оптимального решения, так как проверяются все возможные маршруты.

1.2 Муравьиный алгоритм

Муравьиный алгоритм[2] — это эвристический метод, вдохновлённый поведением колонии муравьёв при поиске оптимальных путей между источниками пищи и муравейником. Алгоритм представляет собой итеративный процесс, в котором множество муравьёв строят решения, используя информацию о феромонах и эвристическую оценку расстояний.

На каждой итерации муравей:

- 1) сформирует 1 маршрут, который претендует на решение задачи коммивояжера на основе вероятностного правила;
- 2) после завершения маршрута обновляется информация о феромонах — на всех рёбрах испаряется часть феромона, а на рёбрах, принадлежащих маршрутам, добавляется новый феромон в зависимости от длины маршрута.

Вероятность перехода определяется количеством феромона на ребре и эвристической полезностью. Итерации продолжаются до выполнения заданного критерия остановки (в данном случае — достижения максимального числа

итераций). Алгоритм не гарантирует нахождения оптимального решения.

Модификация алгоритма по варианту:

- карта перемещения по портам через океаны (время с учётом затрат: на весь маршрут лимитирован объём средств);
- незамкнутый маршрут;
- без элитных муравьев;
- неориентированный граф.

Вывод

В данном разделе были рассмотрены теоретические аспекты алгоритмов решения задачи коммивояжера.

2 Конструкторская часть

В данном разделе будут приведены описания алгоритмов решения задачи коммивояжера, а также модель вычисления трудоемкости алгоритмов.

2.1 Описание алгоритмов

На вход алгоритмов подается информация о графе в виде матрицы связности M . Для муравьиного алгоритма дополнительно задаются следующие параметры:

- максимальное число итераций $t_{\max}$,
- количество муравьёв N_{ants} ,
- коэффициент влияния феромона α ,
- коэффициент испарения феромона ρ ,
- бюджет ресурсов max_resources .

На выходе – длина наилучшего маршрута L_{best} и сам маршрут в виде массива вершин.

На схемах 2.1— 2.6 приведены описания алгоритмов решения задачи коммивояжера: алгоритм полного перебора, муравьиный алгоритм.

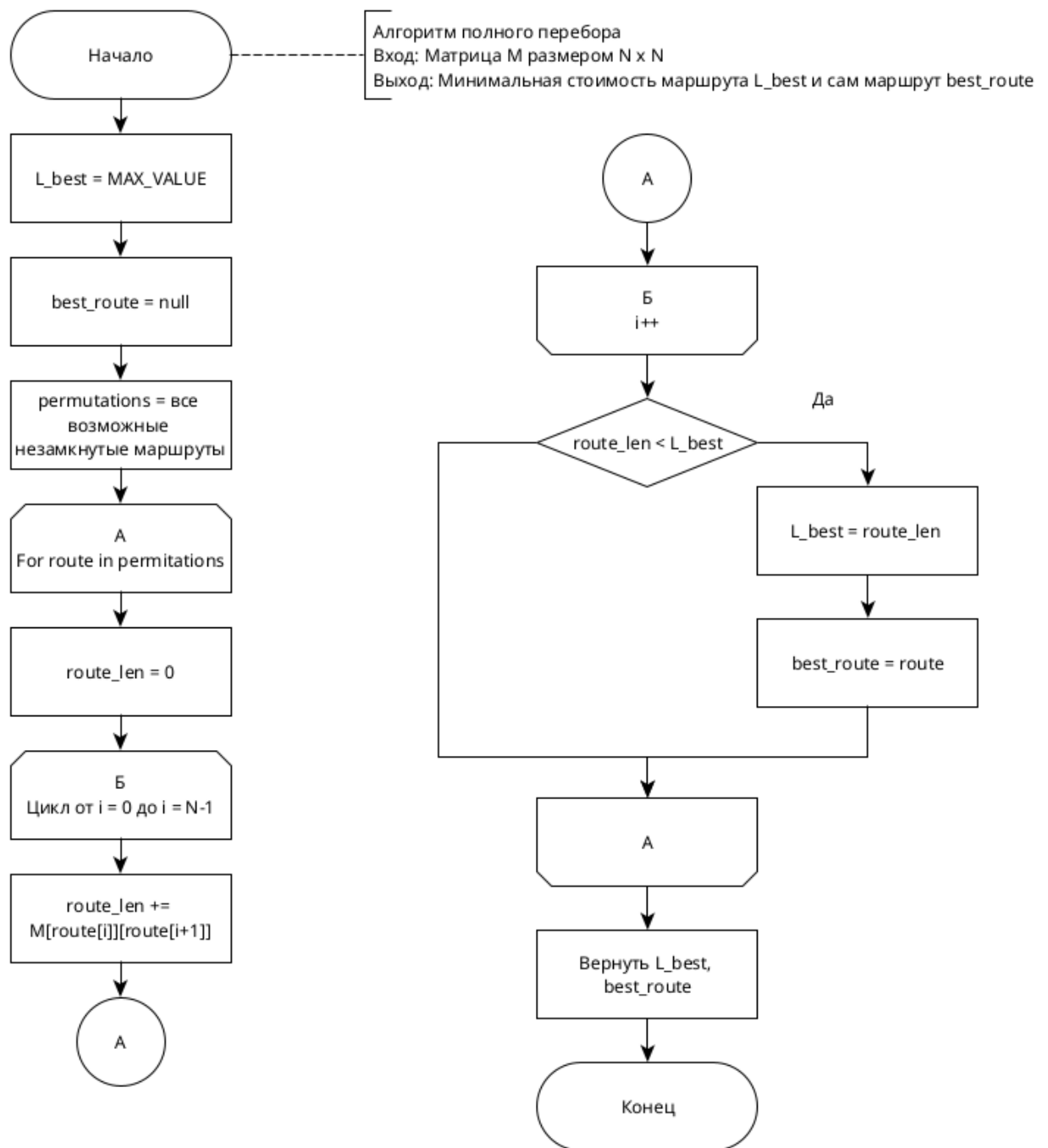


Рисунок 2.1 — Описание алгоритма полного перебора

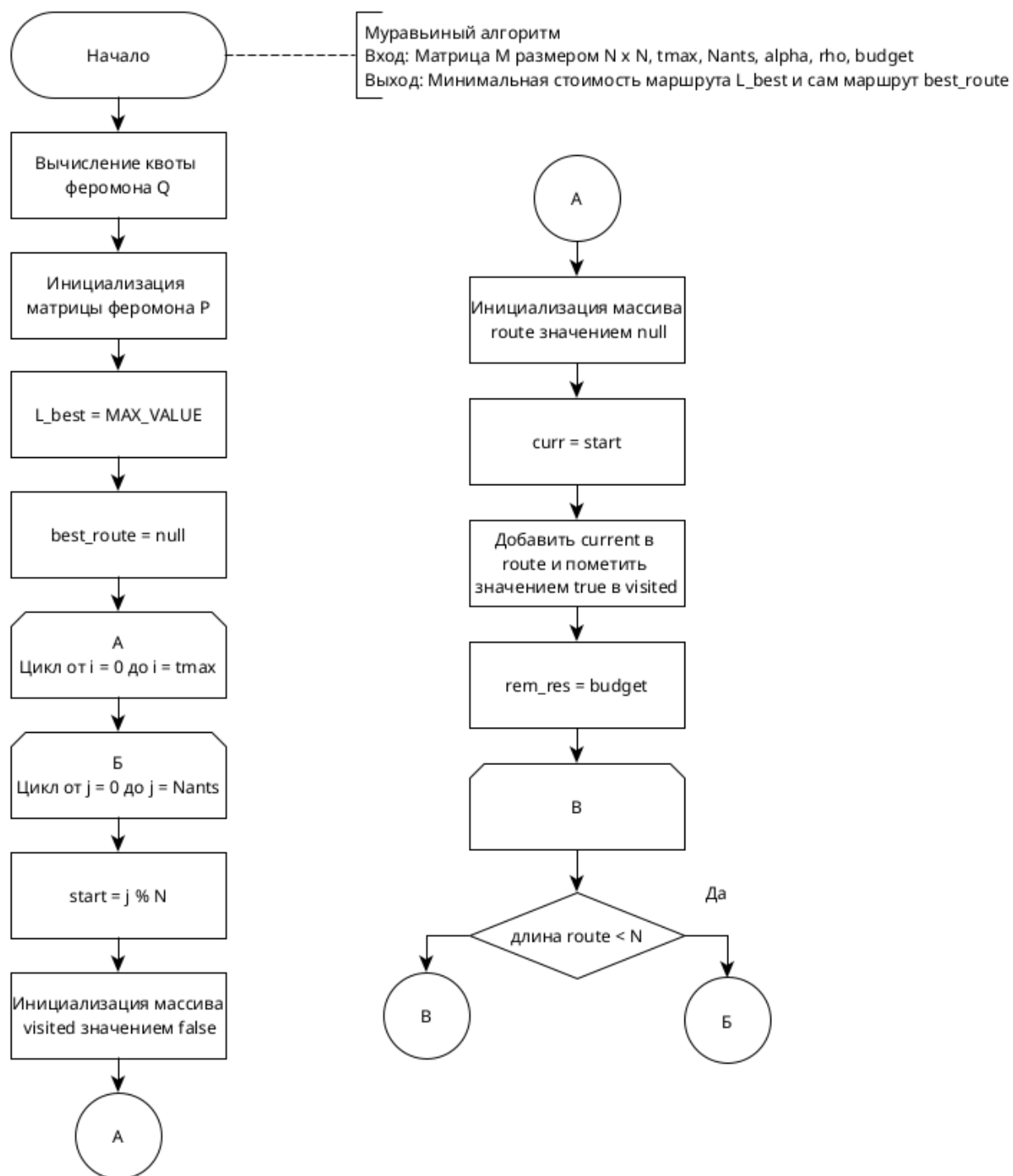


Рисунок 2.2 — Описание муравьиного алгоритма

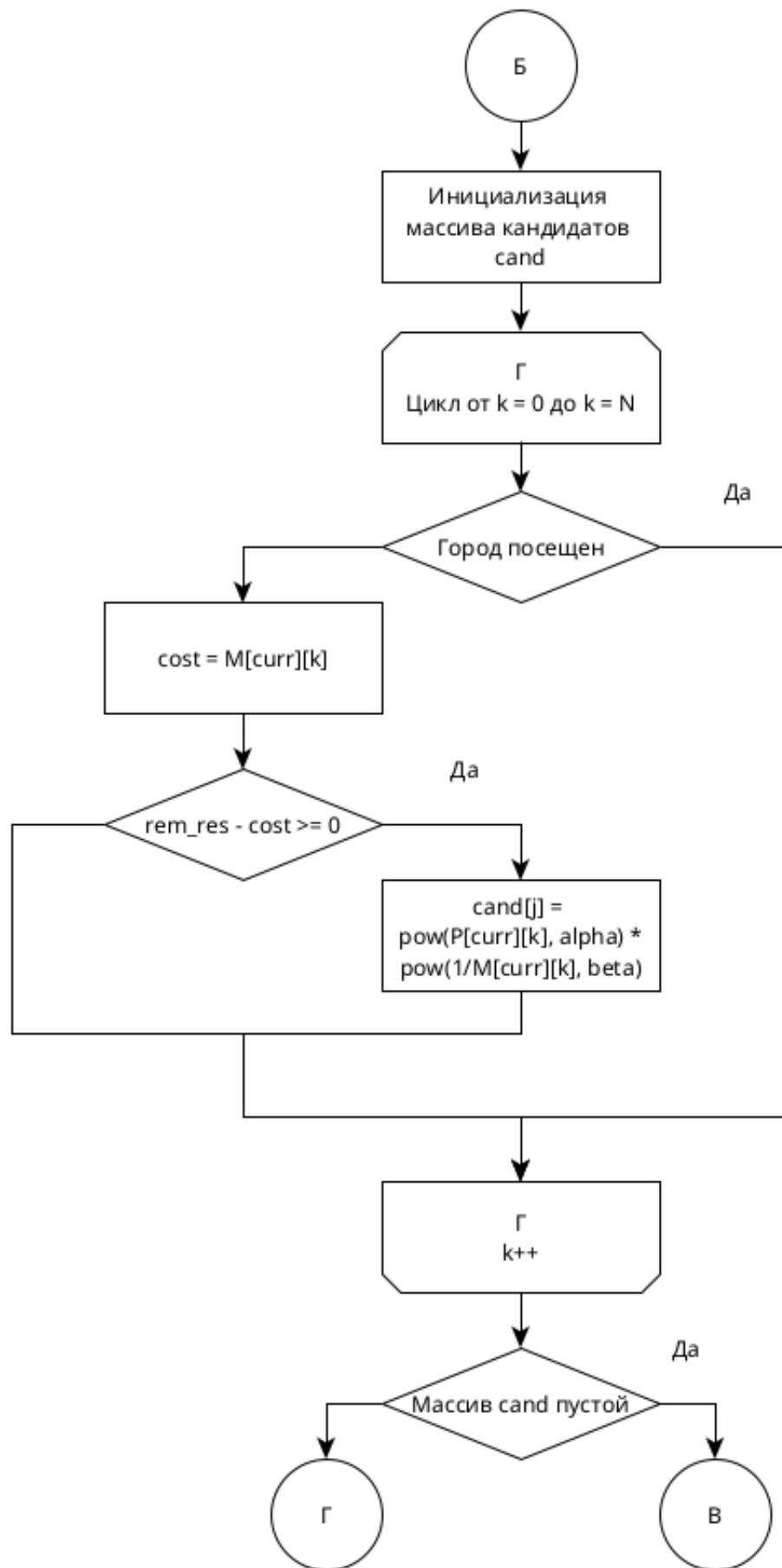


Рисунок 2.3 — Описание муравьиного алгоритма – нахождение вероятностей

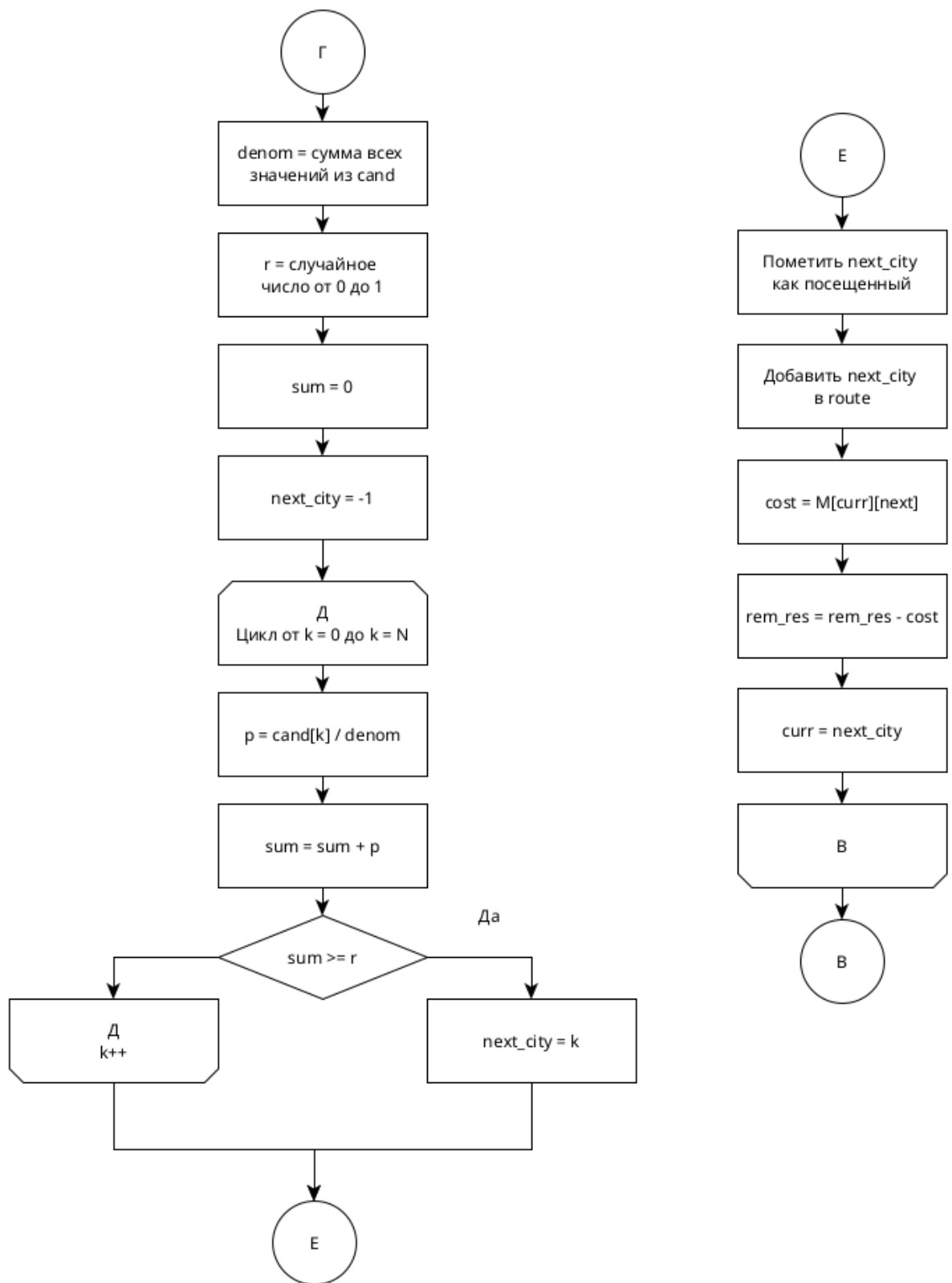


Рисунок 2.4 — Описание муравьиного алгоритма – выбор следующего города

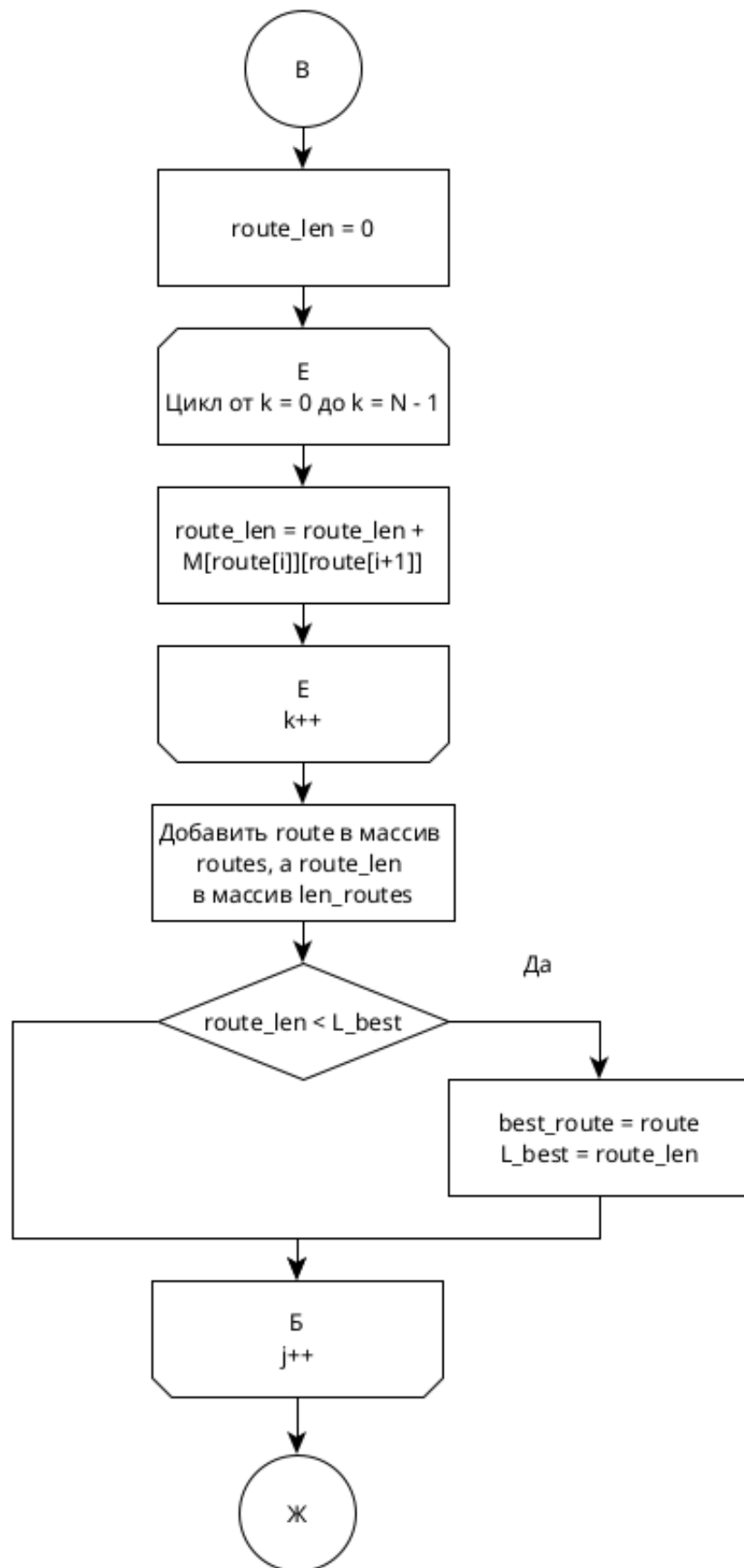


Рисунок 2.5 — Описание муравьиного алгоритма – выбор лучшего решения

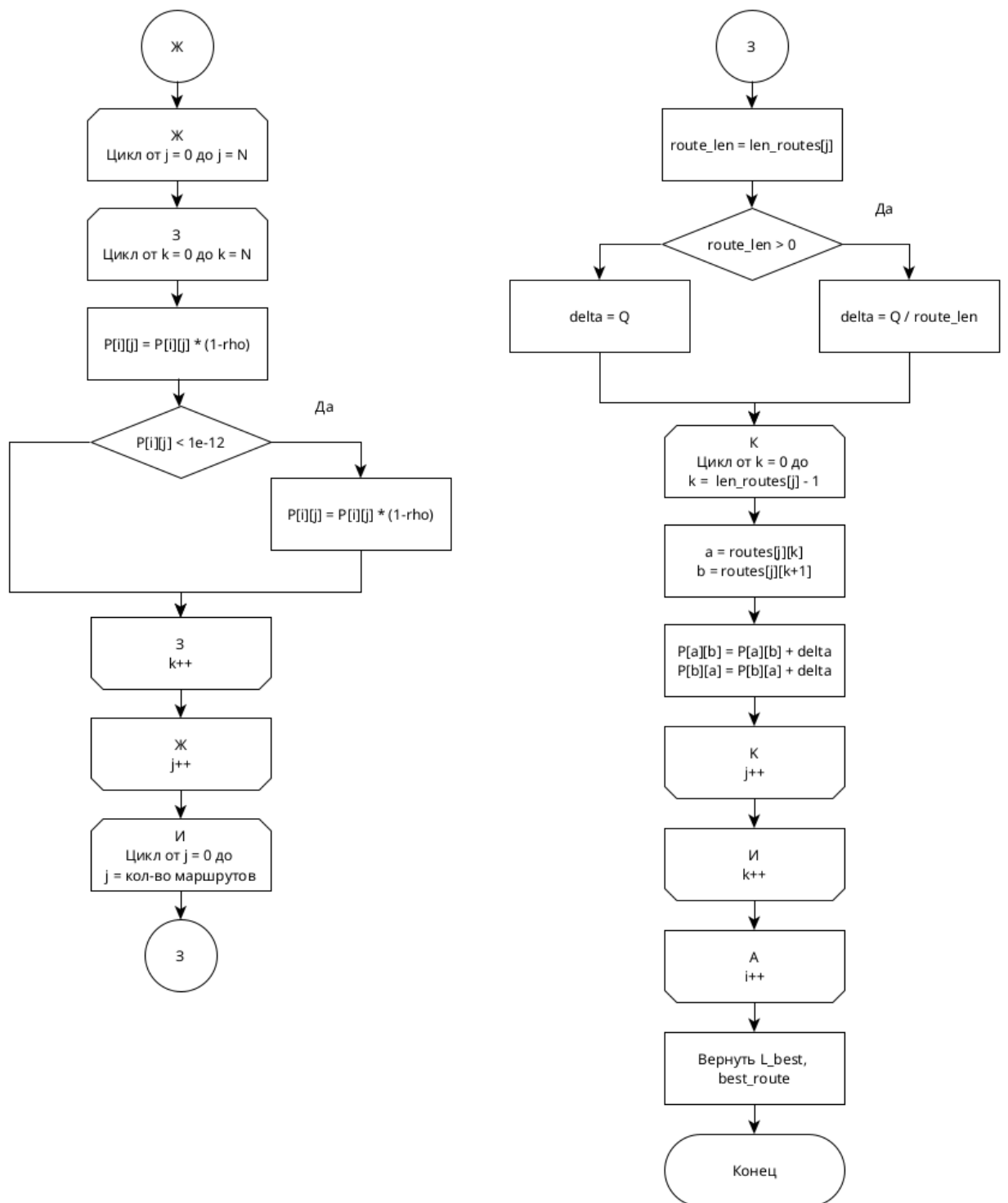


Рисунок 2.6 — Описание муравьиного алгоритма – обновление феромонов

2.2 Модель вычислений

Для вычисления трудоемкостей алгоритмов необходимо ввести модель вычислений:

- операции $+$, $-$, $[]$, $++$, $--$, $==$, $!=$, $<$, $>$, $<=$, $>=$ имеют трудоемкость 1;
- операции $*$, $/$, $\%$, $/=$, $*=$, $\%=$ имеют трудоемкость 2;
- трудоемкость условного оператора `if <condition> then A else B` рассчитывается, как (2.1);

$$f_{if} = f_{condition} + \begin{cases} f_A, & \text{если условие выполняется,} \\ f_B, & \text{иначе.} \end{cases} \quad (2.1)$$

- трудоемкость цикла `for (init; condition; increment) { body }` рассчитывается, как (2.2);

$$f_{for} = f_{init} + f_{condition} + N \cdot (f_{body} + f_{condition} + f_{increment}) \quad (2.2)$$

где N – количество итераций цикла;

- трудоемкость вызова функции равна 0.

2.3 Вычисление трудоемкости алгоритмов

Далее представлен расчет трудоемкостей алгоритмов. Для расчета представлены такие алгоритмы, как алгоритм полного перебора и муравьиный алгоритм.

Проверка входных данных и инициализация массива лучшего маршрута *best_route* и длины *L_{best}* в расчетах учитываться не будут, так как данные действия не являются самыми трудоемкими и являются общими для всех алгоритмов.

Обозначения размерностей:

- N – размерность матрицы смежности;
- M – количество муравьев;
- T_{max} – количество итераций.

2.3.1 Алгоритм полного перебора

Расчет трудоемкости стандартного алгоритма:

- трудоемкость генерации всех перестановок, $f_{perm} = N! \cdot N$;
- трудоемкость внешнего цикла по перестановкам, $f_i = 3 + N! \cdot (2 + f_{body_i})$;
- трудоемкость цикла по вершинам графа, $f_j = 3 + (N - 1) \cdot (3 + f_{sum})$;
- трудоемкость суммирования меток, $f_{sum} = 7$.

Трудоемкость проверки на лучший маршрут 2.3:

$$f_{cond} = \begin{cases} 1, & \text{длина маршрута больше или равна лучшего(л.с.),} \\ 3, & \text{иначе (х.с.).} \end{cases} \quad (2.3)$$

Итоговая трудоемкость:

$$f_{brf} = N! \cdot N + 3 + N!(2 + 1 + 3 + (N - 1) \cdot 7 + 1) - \text{л.с.}$$

$$f_{brf} = N! \cdot N + 3 + N!(2 + 1 + 3 + (N - 1) \cdot 7 + 3) - \text{х.с.}$$

После приведения:

$$f_{brf} = \begin{cases} N! \cdot 8N + 3, & (\text{л.с.}), \\ N! \cdot (8N + 2) + 3, & (\text{х.с.}), \end{cases} \quad (2.4)$$

2.3.2 Муравьиный алгоритм

Расчет трудоемкости муравьиного алгоритма:

- сложность инициализации начальных параметров пренебрежимо мала, $f_1 = 1$;
- инициализация квоты феромона $F_Q = 2N$;
- инициализация матрицы феромонов $F_P = N^2$;
- инициализация эвристической матрицы $F_{eta} = N^2$;
- сложность основного цикла по итерациям $f_{main} = 2 + T_{max} \cdot (2 + f_{bodyT})$;
- сложность внутреннего цикла по муравьям $f_{ants} = 2 + M \cdot (2 + f_{bodyA})$.

Расчет сложности по телу цикла f_{ants} , инициализация:

$$f_2 = 3 + N + 2 + 2 + 1 + 1 = 9 + N.$$

Поиск вероятностей:

$$f_3 = 2 + N \cdot (2 + N \cdot (2 + 2 + 3 + 2 + 11 + 2) + 1 + N + 2 + 1 + 1 + f_{nextC}) =$$

$$2 + N \cdot (23N + 7 + f_{nextC}).$$

Поиск следующего города:

$$f_{nextC} = 2 + \frac{N}{2}(2 + 3 + 3 + 2) + 2 + 2 + 3 + 2 + 1 = 5N + 12.$$

Суммарная сложность цикла обхода городов:

$$f_3 = N + 2 + N \cdot (23N + 7 + 5N + 12) = 27N^2 + 20N + 2.$$

Обновление результата:

$$f_{upd} = 1 + 7(N - 1) + 4 + 2 = 7N$$

Суммарная сложность цикла по муравьям:

$$f_{ants} = 2 + M \cdot (2 + f_2 + f_3 + f_{upd}) = 2 + M \cdot (2 + 9 + N + 27N^2 + 20N + 2 + 7N) = 2 + M \cdot (27N^2 + 28N + 13)$$

Обновление феромонов:

$$f_{pher} \approx N^2$$

Нанесение феромонов:

$$f_{pher2} \approx N * M$$

Результирующая трудоемкость муравьиного алгоритма:

$$f_{Aans} = 1 + 2N + N^2 + N^2 + 2 + T_{max} \cdot (2 + 2 + M \cdot (27N^2 + 28N + 13) + N^2 + N * M) \approx 27T_{max} \cdot MN^2 \approx 27N^4$$

Вывод

В данном разделе были представлены схемы алгоритмов решения задачи коммивояжера, а также оценены трудоемкости для данных алгоритмов. Алгоритм полного перебора имеет большую трудоемкость по сравнению с муравьиным.

3 Технологическая часть

В этом разделе обоснованы средства реализации, а так же представлены реализации алгоритмов и функциональные тесты.

3.1 Средства реализации

Для реализации алгоритмов в этой лабораторной работы был выбран язык *Python*^[3] по следующим причинам:

- python предоставляет необходимые структуры данных, такие как списки (массивы) и вложенные списки (матрицы);
- поддерживаемые библиотеки, такие как *matplotlib*^[4] позволяют визуализировать результаты исследования.

3.2 Реализация алгоритмов

В листингах 3.1 — 3.2 представлены реализации алгоритмов.

Листинг 3.1 — Алгоритм полного перебора

```
def generate_permutations(arr):
    result = []
    def permute(a, l):
        if l == len(a):
            result.append(a[:])
        else:
            for i in range(l, len(a)):
                a[l], a[i] = a[i], a[l]
                permute(a, l+1)
                a[l], a[i] = a[i], a[l]
    permute(arr, 0)
    return result

def tsp_brute_force(distance_matrix):
    n = len(distance_matrix)
    vertices = list(range(n))

    all_permutations = generate_permutations(vertices)

    best_length = float('inf')
    best_route = None

    for route in all_permutations:
        route_length = 0.0
        for i in range(len(route)-1):
            route_length += distance_matrix[route[i]][route[i+1]]
        if route_length < best_length:
            best_length = route_length
            best_route = route[:]

    return best_route, best_length
```

Листинг 3.2 — Муравьиный алгоритм

```
def ant_colony_tsp_with_resources(distance_matrix,
                                  num_ants=10,
                                  num_iterations=100,
                                  alpha=0.5,
                                  beta=0.5,
                                  rho=0.5,
                                  tau_0=0.01,
                                  max_resources=100.0):

    n = len(distance_matrix)

    total_dist = 0.0
    for i in range(n):
        for j in range(n):
            if i != j:
                total_dist += distance_matrix[i][j]
    Q = total_dist / n

    pheromone = [[tau_0 for _ in range(n)] for _ in range(n)]
    eta = [[0 if distance_matrix[i][j] == 0 else 1.0 / distance_matrix[i][j]
            for j in range(n)] for i in range(n)]

    best_route = None
    best_length = float('inf')

    for iteration in range(num_iterations):
        all_routes = []
        all_lengths = []

        for ant in range(num_ants):
            start = ant % n
            visited = [False] * n
            visited[start] = True
            route = [start]
            current = start

            remaining_resources = max_resources

            while len(route) < n:

                candidates = []
                for j in range(n):
                    if not visited[j]:
                        cost = distance_matrix[current][j]
                        if remaining_resources -- cost >= 0:
                            val = (pheromone[current][j] ** alpha) * (eta[
```

```

        current][j] ** beta)
        candidates.append((j, val))

    if len(candidates) == 0:
        break

    denom = sum(val for _, val in candidates)
    r = random.random()
    cumulative = 0.0
    next_city = None
    for (city, val) in candidates:
        p = val / denom
        cumulative += p
        if r <= cumulative:
            next_city = city
            break

    visited[next_city] = True
    route.append(next_city)

    cost = distance_matrix[current][next_city]
    remaining_resources -= cost
    current = next_city

route_length = 0.0
for i in range(len(route) - 1):
    route_length += distance_matrix[route[i]][route[i + 1]]

all_routes.append(route)
all_lengths.append(route_length)

if route_length < best_length:
    best_length = route_length
    best_route = route[:]

for i in range(n):
    for j in range(n):
        pheromone[i][j] *= (1 - rho)
        if pheromone[i][j] < 1e-12:
            pheromone[i][j] = 1e-12

for k, route in enumerate(all_routes):
    route_length = all_lengths[k]
    if route_length > 0:
        delta = Q / route_length

```

```

else:
    delta = Q
    for i in range(len(route) - 1):
        a, b = route[i], route[i + 1]
        pheromone[a][b] += delta
        pheromone[b][a] += delta

return best_route, best_length

```

3.3 Тестирование

При тестировании муравьиного алгоритма нельзя проверить при помощи сравнения с точным значением, в отличие от полного перебора. Можно лишь убедиться, что результирующий путь содержит все вершины.

Тест 1

Входные данные:

$$\begin{bmatrix}
 0 & 2 & 9 & 10 & 5 \\
 1 & 0 & 6 & 4 & 7 \\
 15 & 7 & 0 & 8 & 3 \\
 6 & 3 & 12 & 0 & 11 \\
 9 & 5 & 2 & 6 & 0
 \end{bmatrix}$$

Ожидаемый лучший маршрут: [3, 1, 0, 4, 2], длина = 11

Выходные данные полного перебора: [3, 1, 0, 4, 2], длина = 11

Выходные данные муравьиного алгоритма: [3, 1, 0, 4, 2], длина = 11

Тест 2

Входные данные:

0	8	4	1	9	27	5	8	12	14
8	0	13	4	14	12	13	7	20	4
4	13	0	5	16	8	18	4	21	17
1	4	5	0	7	12	9	8	13	16
9	14	16	7	0	12	17	12	17	4
27	12	8	12	12	0	8	13	9	4
5	13	18	9	17	8	0	10	13	1
8	7	4	8	12	13	10	0	7	11
12	20	21	13	17	9	13	7	0	10
14	4	17	16	4	4	1	11	10	0

Ожидаемый лучший маршрут: [1, 3, 0, 2, 7, 8, 5, 6, 9, 4], длина = 42

Выходные данные полного перебора: [1, 3, 0, 2, 7, 8, 5, 6, 9, 4], длина = 42

Выходные данные муравьиного алгоритма: [4, 9, 6, 0, 3, 1, 7, 2, 5, 8], длина = 43

Вывод

В технологической части были представлены листинги реализованных алгоритмов, включая алгоритм полного перебора и муравьиный алгоритм.

4 Исследовательская часть

В этом разделе приведен сравнительный анализ алгоритмов по времени выполнения.

4.1 Технические характеристики ЭВМ

Все замеры проводились на ЭВМ, характеристики которой приведены ниже:

- процессор: AMD Ryzen™ 7–5800H with Radeon™ Graphics × 16;
- ОС: 64-разрядная операционная система Ubuntu 24.04.1 LTS;
- оперативная память: 16,0 ГБ.

4.2 Сравнение алгоритмов

В таблице 4.1 приведено время работы алгоритмов в секундах. Для повышения точности результатов проводилось 50 замеров.

Таблица 4.1 — Сравнение алгоритмов решения задачи коммивояжера по времени выполнения

Размерность матрицы	Полный перебор (сек)	Муравьиный алгоритм (сек)
2	0.00000	0.00015
3	0.00001	0.00027
4	0.00002	0.00044
5	0.00010	0.00062
6	0.00106	0.00083
7	0.00622	0.00113
8	0.05782	0.00137

На графике 4.1 приведена визуализация результатов.

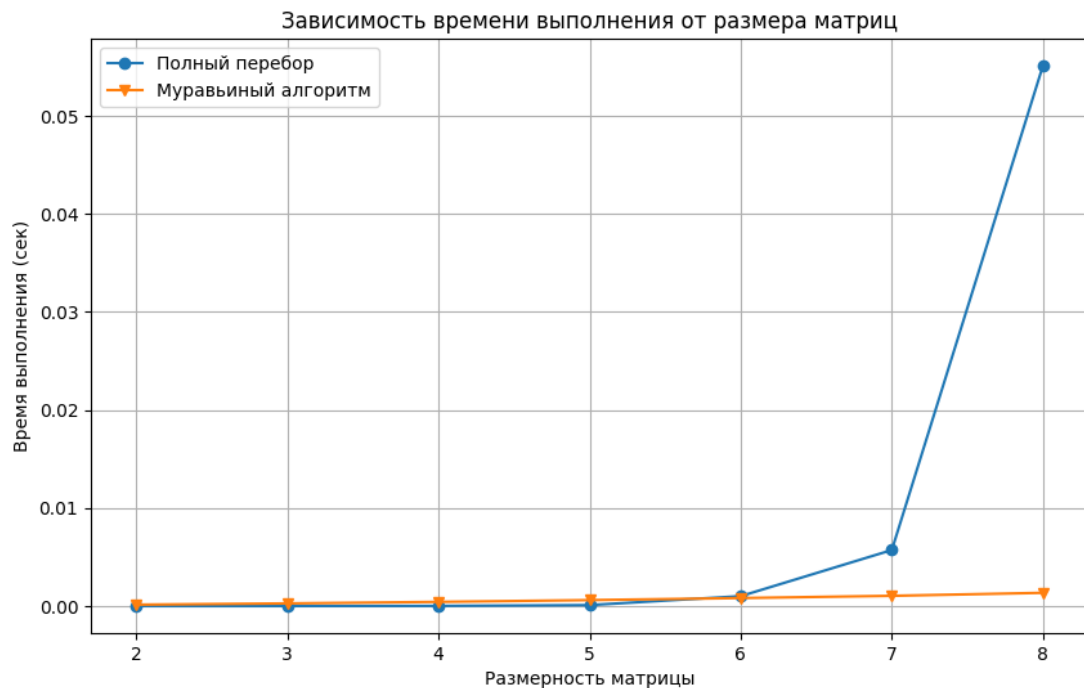


Рисунок 4.1 — Сравнения для каждого элемента в алгоритме полного перебора

4.3 Параметризация

Муравьиный алгоритм не гарантирует нахождение оптимального решения задачи. Целью проведения параметризации является определение параметров, при которых муравьиный алгоритм дает наилучшие результаты.

Для параметризации в качестве критерия качества получаемого решения используется максимальное значение, медианное и среднее арифметическое значения отклонения затрат полученного маршрута от эталонных затрат.

Одно медианное значение, как и одно максимальное, как и одно среднее арифметическое, — каждое получается для серии результатов для каждого графа, для одного графа выполнено 10 запусков поиска решения муравейником.

4.3.1 Класс данных

В качестве класса данных для параметризации используются графы, построенные на портах Атлантического, Тихого и Индийского океанов. Всего 3 графа, представленные матрицами смежности размером 10 на 10. Расстояния

представлены в километрах. Сами расстояния были найдены при помощи приложения *Google Maps*[5].

Атлантический океан

Порты:

- 1) Нью Йорк (США);
- 2) Норфолк (США);
- 3) Чарльстон (США);
- 4) Гамбург (Германия);
- 5) Роттердам (Нидерланды);
- 6) Ле Хавре (Франция);
- 7) Ливерпуль (Великобритания);
- 8) Касабланка (Марокко);
- 9) Буэнос-Айрес (Аргентина);
- 10) Сантос (Бразилия).

0	600	1100	6600	6500	6200	5300	5800	8400	7600
600	0	630	6700	6600	6300	5400	6000	8400	7800
1100	630	0	7200	7100	6800	5900	6200	8000	7400
6600	6700	7200	0	600	900	960	2900	11700	10600
6500	6600	7100	600	0	600	850	2800	11600	10500
6200	6300	6800	900	600	0	700	2400	11500	10400
5300	5400	5900	960	850	700	0	2200	11400	10300
5800	6000	6200	2900	2800	2400	2200	0	8100	7600
8400	8400	8000	11700	11600	11500	11400	8100	0	1900
7600	7800	7400	10600	10500	10400	10300	7600	1900	0

Тихий океан

Порты:

- 1) Шанхай (Китай);
- 2) Пусан (Республика Корея);
- 3) Йокохама (Япония);

- 4) Лос Анджелес (США);
- 5) Long Beach (США);
- 6) Oakland (США);
- 7) Ванкувер (Канада);
- 8) Сиэтл (США);
- 9) Манила (Филиппины);
- 10) Гонконг (Китай).

0	870	1820	10400	10450	9850	8900	9000	1840	1260
870	0	1100	9840	9900	9450	8200	8350	2000	2100
1820	1100	0	8800	8900	8300	7500	7600	3000	2900
10400	9840	8800	0	30	560	1700	1550	11700	11600
10450	9900	8900	30	0	640	1750	1600	11800	11700
9850	9450	8300	560	640	0	1300	1200	11500	11400
8900	8200	7500	1700	1750	1300	0	230	10500	10400
9000	8350	7600	1550	1600	1200	230	0	10400	10300
1840	2000	3000	11700	11800	11500	10500	10400	0	1500
1260	2100	2900	11600	11700	11400	10400	10300	1500	0

Индийский океан

Порты:

- 1) Мумбай (Индия);
- 2) Коломбо (Шри-Ланка);
- 3) Карачи (Пакистан);
- 4) Дербин (ЮАР);
- 5) Mombasa (Кения);
- 6) Perth (Австралия);
- 7) Порт Лоис (Маврикий);
- 8) Salalah (Оман);
- 9) Дубай (ОАЭ);
- 10) Сингапур (Сингапур).

0	1500	900	6200	4600	7800	4700	2400	1920	3900
1500	0	2400	5300	3600	6700	3000	3100	3300	3300
900	2400	0	6400	4200	7600	5300	2450	1200	5000
6200	5300	6400	0	2400	6500	3000	5600	6600	7100
4600	3600	4200	2400	0	6500	1900	2500	3600	6300
7800	6700	7600	6500	6500	0	5800	8000	8750	3900
4700	3000	5300	3000	1900	5800	0	4600	5300	4500
2400	3100	2450	5600	2500	8000	4600	0	1000	5600
1920	3300	1200	6600	3600	8750	5300	1000	0	5800
3900	3300	5000	7100	6300	3900	4500	5600	5800	0

4.3.2 Результат параметризации

В результате параметризации получена таблица со следующими параметрами:

- α – коэффициент влияния феромона;
- ρ – коэффициент испарения феромона;
- дни – число итераций;
- Max , Med , Avg – максимальное, медианное и среднее значение отклонения.

Далее представлен фрагмент из таблицы с наилучшими результатами 4.2. Полная таблица представлена в приложении А.

Таблица 4.2 — Результаты параметризации муравьиного алгоритма (начало)

Параметры			Граф 1			Граф 2			Граф 3		
α	ρ	Дни	max	med	avg	max	med	avg	max	med	avg
0.5	0.1	200	3.4	1.1	1.3	4.0	1.4	1.6	39.0	23.0	22.0
0.5	0.1	300	6.0	1.2	2.3	2.0	0.0	0.4	32.0	21.0	16.7
0.5	0.1	400	2.4	0.4	0.8	2.0	0.0	0.3	30.0	16.6	15.5
0.5	0.1	500	3.0	1.9	1.4	0.8	0.0	0.2	21.0	10.0	9.6
0.5	0.25	100	6.4	3.4	3.2	7.8	2.0	2.4	44.0	29.6	26.0

Вывод

В результате исследования по времени было установлено, что при размерности матрицы ≤ 6 , муравьиный алгоритм и алгоритм полного перебора выполняют задачу примерно за одинаковое время. При большей размерности (7 и более) муравьиный алгоритм работает примерно в 6 раз быстрее.

Результаты параметризации показали, что наилучшей комбинацией параметров для муравьиного алгоритма будет $\alpha = 0.5$, $\rho = 0.1$, $t_{max} = 500$.

ЗАКЛЮЧЕНИЕ

В ходе работы были исследованы два алгоритма для решения задачи коммивояжёра: полный перебор и муравьиный алгоритм. Каждый из них имеет свои преимущества и ограничения, которые определяют их применимость в различных ситуациях.

Метод полного перебора обеспечивает нахождение оптимального решения, поскольку он проверяет все возможные перестановки маршрутов. Однако из-за факториальной сложности его использование ограничено задачами с числом городов ≤ 6 .

Муравьиный алгоритм представляет собой эвристический метод, который вдохновлён поведением муравьёв в природе. Этот алгоритм обеспечивает приемлемые результаты для графов с 7 и более вершин. Несмотря на то, что он не гарантирует нахождение оптимального решения, получаемые маршруты близки к оптимальным. Эффективность алгоритма также зависит от его параметров. Результат исследования показал, что оптимальным набором является $\alpha = 0.5$, $\rho = 0.9$, $t_{max} = 500$.

В ходе выполненной работы были выполнены следующие задачи:

- описаны стандартный метод решения задачи коммивояжера и метод на базе муравьиного алгоритма;
- описана модель вычислений;
- выполнена оценка трудоемкости разрабатываемых алгоритмов;
- реализованы алгоритмы;
- проведен сравнительный анализ двух рассмотренных методов решения задачи коммивояжера;
- выполнена параметризация метода, основанного на муравьином алгоритме по трём его параметрам.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. В. И. Мудров Задача о коммивояжере. - Москва: Знание, 1969. - 64 с.
2. Пападимитриу Х., Стайглиц К. Комбинаторная оптимизация. Алгоритмы и сложность. - Нью Джерси: PRENTICE-HALL, 1982. - 512 с.
3. Welcome to Python – [Электронный ресурс]. URL: <https://https://www.python.org> (дата обращения: 25.11.2024).
4. Библиотека matplotlib – [Электронный ресурс]. URL: <https://matplotlib.org/> (дата обращения: 25.11.2024).
5. Google Maps – [Электронный ресурс]. URL: <https://www.google.com/maps/> (дата обращения: 15.12.2024).

ПРИЛОЖЕНИЕ А

Результаты параметризации